Introduction

Assignment: Assignment 2
Authors: Weiming Wu, Jazeps Augustans

The assignment evaluates Pearson correlation and radius-15 Gaussian blur, which have different workloads. Pearson calculates correlations for every unique vector pair, while Blur applies horizontal and vertical filtering to each RGB channel. The objectives were to establish a reproducible baseline, identify hotspots remaining after compiler optimization, apply measurable source-level optimizations, verify output correctness, and explain the results using timing and profiler evidence.

System Specifications

Measurements were performed inside a KVM virtual machine (AMD Ryzen 7 9800X3D, 8 vCPUs, 15 GiB RAM, swap disabled) running Ubuntu 25.04. GCC 15.2.0 was used for blur and GCC 13.4.0 (g++-13) for Pearson. The virtualised environment introduces host-scheduling variance in elapsed and CPU-time measurements.

Tools, Method, and Limitations

GNU time (/usr/bin/time -v) measured elapsed, user, and system time, CPU utilization, and maximum RSS in KiB (%M). **Valgrind Callgrind** measured instruction and data references, function costs, cache misses, branches, and branch mispredictions using --cache-sim=yes and --branch-sim=yes while **GNU gprof** collected sampled function times and call counts from executables compiled with -pg. Pearson outputs were checked with verify, and Blur outputs were compared byte-for-byte with reference images using cmp.

All programs used –02 flag. Each input was measured in five batches, with executions doubled until each batch lasted about two seconds. Results were normalized per execution: the mean used total time divided by total executions, and the median used the five normalized batches. A one-second pause separated batches. RSS and correctness were measured separately. gprof used 1024.data and im4.ppm; Callgrind used 512.data and im2.ppm.

Testing in a virtual machine caused unstable CPU accounting for short runs, so CPU utilization was treated only as supporting evidence. gprof also collected only 50 Pearson samples and eight Blur samples, making its timing percentages unreliable. Its call counts remained useful, but Callgrind was used for hotspot percentages.

Baseline Performance

Blur

Image	Median wall (s)	User (s)	Sys (s)	CPU %	Max RSS (KB)	Correct
im1.ppm	0.228	0.156	0.032	84.13	16,960	PASS
im2.ppm	0.450	0.272	0.032	67.52	25,604	PASS
im3.ppm	0.770	0.561	0.075	84.28	37,232	PASS
im4.ppm	3.890	3.336	0.404	91.94	135,996	PASS

Figure 1. Execution time and CPU utilization

Wall time grows with pixel count across all four images, and CPU utilization is high on every run (67–92%), indicating a CPU-bound workload. Median wall-time goes from 0.228 on im1 up to 3.89 second on im4. The lower utilization on im2 is likely due to process startup overhead being larger on the smaller image. Peak RSS is 16.6 MB on im1 despite only ~0.2 MB of pixel data, because the baseline allocates a fixed 3000×3000 scratch buffer regardless of input size.

Function	Ir	Dr	Dw	% Ir
`Filter::blur`	5,353,877,552	1,042,677,770	2,791,577,613	59.02%
`__ieee754_exp_fma`	1,061,158,912	350,224,384	0	11.70%
`exp@@GLIBC_2.29`	637,534,208	167,772,160	100,663,296	7.03%
`Matrix::r(unsigned, unsigned)`	333,086,720	199,852,032	0	3.67%
`Matrix::b(unsigned, unsigned)`	333,086,720	199,852,032	0	3.67%
`Matrix::g(unsigned, unsigned)`	333,086,720	199,852,032	0	3.67%

Figure 2. baseline Callgrind profiler results

The profiler, Figure 2, shows three specific problems in the baseline blur. First, 59% of all instructions are inside the pixel loops themselves. Second, nearly 20% is spent recomputing the same 16 Gaussian weight values on every pixel, even though those values only depend on the radius. Third, another 11% goes to Matrix::r/g/b accessor calls that do a multiply and a memory load for every single pixel read.

Pearson

Input	Executions	Mean elapsed (s)	Median elapsed (s)	User (s)	System (s)	CPU	Max RSS (MiB)	Correct
128	1,000	0.01972	0.01930	0.01217	0.01465	136.00%*	4.61	Pass
256	200	0.09430	0.09325	0.04195	0.01470	60.07%*	5.29	Pass
512	15	0.57267	0.57667	0.47733	0.07933	97.21%	9.21	Pass
1024	5	3.58000	3.56000	3.18600	0.32200	97.99%	24.22	Pass

*CPU accounting was unstable for the two shortest inputs and is not used for conclusions.

Figure 3. Pearson -02 baseline. Each input was measured in five batches.

Mean time increased approximately 182x from 128.data to 1024.data; the two largest doublings each increased time by approximately 6.1x. Mean and median times remained close for every input. The larger inputs occupied approximately one CPU core, consistent with sequential execution. For 1024.data, user and system time totaled 3.508 seconds versus 3.580 seconds mean elapsed time. RSS increased with dataset size, and every output passed the supplied verifier.

Function	Callgrind instructions	Instruction share
`Vector::operator-(double)`	1,481,360,384	21.42%
`Vector::operator/(double)`	1,481,360,384	21.42%
`Vector::Vector(const Vector&)`	1,017,264,106	14.71%
`Vector::magnitude()`	813,675,520	11.77%
`Vector::dot(Vector)`	403,436,544	5.83%
`Vector::mean()`	338,551,808	4.90%

Figure 4. Pearson -02 basline Callgrind profiler results

Optimization 1

Blur

The baseline recomputed the Gaussian weights inside both pixel loops. The weights depend only on radius. Yet get_weights was called once per pixel per pass. Therefore, the optimization, that is, Loop-invariant code motion, was applied to compute the weights once, before both loops.

Image	Baseline user (s)	Opt 1 user (s)	CPU speedup	Baseline RSS (KB)	Opt 1 RSS (KB)	Correctness
im1.ppm	0.156	0.088	1.77×	16,960	16,864	IDENTICAL
im2.ppm	0.272	0.221	1.23×	25,604	25,468	IDENTICAL
im3.ppm	0.561	0.410	1.37×	37,232	37,304	IDENTICAL
im4.ppm	3.336	2.304	1.45×	135,996	135,928	IDENTICAL

Figure 5. Optimization 1 — User CPU, Memory, Correctness vs baseline

Function	Ir	Dr	Dw	% Ir
`Filter::blur`	2,713,683,202	813,535,310	591,909,915	60.43%
`Matrix::r(unsigned, unsigned)`	333,086,720	199,852,032	0	7.42%
`Matrix::b(unsigned, unsigned)`	333,086,720	199,852,032	0	7.42%
`Matrix::g(unsigned, unsigned)`	333,086,720	199,852,032	0	7.42%
`std::basic_streambuf::xsgetn`	125,829,131	28,311,559	25,165,825	2.80%

Figure 6. optimization 1 callgrind profiler results

In the baseline, two functions — __ieee754_exp_fma and exp@@GLIBC_2.29 — together account for 1,698,693 or 18.73% of the entire program. In the Figure 6, both are completely gone from top rows. Inside Filter::blur itself, instructions drop from 5,353,877,552 to 2,713,683,202, a 49% reduction compared to baseline callgrind . This is the removal of the double w[Gauss::max_radius]{} . User CPU time drops 30–44% across every image,based on Figure 5, consistent with the 1.45× median wall time speedup. Peak memory is unchanged as expected.

From the Callgrind totals, branches executed dropped from 2,477,110,453 in the baseline to 175,485,645 in Optimization 1, a reduction of 92.9%. The Gaussian weight loop for (i = 0; i <= radius; i++) was running once per pixel per pass. Optimzation 1 runs that same loop once total, so almost all of those branches disappear.

Pearson

Callgrind identified subtraction, division, vector copying, magnitude, and mean calculations as the main instruction hotspots. As shown in Figure 4, Vector::operator-(double), Vector::operator/(double), Vector::magnitude() const, and Vector::mean() const together accounted for **59.51% of the baseline instruction count**.

Source inspection revealed that correlation_coefficients() calls pearson() for every unique vector pair, repeatedly copying, centering, and normalizing the same vectors. Since each vector's

normalized form remains unchanged across comparisons, **Optimization 1 applies loop-invariant computation removal** by normalizing each vector only once.

A new void Vector::normalize() function performs in place mean subtraction and magnitude based scaling, eliminating temporary vectors created by the subtraction and division operators. The correlation_coefficients() function now normalizes each dataset before the pairwise loops and computes correlations using Vector::dot().

Because the dot product of two mean-centered, unit-length vectors equals their Pearson correlation coefficient, this approach preserves the original calculation, pair ordering, and clamping to [-1, 1], while eliminating redundant normalization and temporary-vector creation.

Input	Baseline user (s)	Optimization 1 user (s)	Baseline system (s)	Optimization 1 system (s)	Baseline CPU	Optimization 1 CPU	Baseline RSS (MiB)	Optimization 1 RSS (MiB)	Correctness
128.data	0.01217	0.01402	0.01465	0.00909	136.00%*	152.14%*	4.61	4.63	PASS
256.data	0.04195	0.02540	0.01470	0.02355	60.07%*	99.59%*	5.29	5.25	PASS
512.data	0.47733	0.07450	0.07933	0.04450	97.21%	56.98%*	9.21	9.17	PASS
1024.data	3.18600	0.57200	0.32200	0.35400	97.99%	93.91%	24.22	24.25	PASS

*CPU utilization was unstable for several shorter measurements and is included only as supporting data. It was not used to calculate speedup.

Figure 7. Comparison of optimization 1 vs baseline on resources and correctness

Input	Baseline executions	Optimization 1 executions	Baseline mean (s)	Optimization 1 mean (s)	Baseline median (s)	Optimization 1 median (s)	Mean speedup	Median speedup	Mean time reduction
128.data	1,000	1,000	0.01972	0.01519	0.01930	0.01540	1.30x	1.25x	23.0%
256.data	200	200	0.09430	0.04915	0.09325	0.04825	1.92x	1.93x	47.9%
512.data	15	60	0.57267	0.20883	0.57667	0.19917	2.74x	2.90x	63.5%
1024.data	5	10	3.58000	0.98600	3.56000	0.94000	3.63x	3.79x	72.5%

Figure 8. Comparison of Optimization 1 timings vs baseline timings

Optimization 1 was compared with the baseline reduced mean runtime for every workload, with speedup increasing from 1.30x on 128.data to 3.63x on 1024.data as pairwise computation became more dominant.

For 512.data, the optimized mean and median were 0.209 and 0.199 seconds. For 1024.data, most batches took 0.875–0.960 seconds, while one slower batch took 1.240 seconds and increased the mean. Combined user and system CPU time on 1024.data fell by 73.6%, from 3.508 to 0.926 seconds. Peak RSS remained nearly unchanged, confirming that the optimization reduced repeated computation without increasing maximum memory use. Unstable CPU-utilization values were not used to calculate speedup.

Function reported by Callgrind	Instructions	Share of all instructions
`Vector::dot(Vector) const`	403,436,544	24.25%
`Vector::Vector(Vector const&)`	341,730,282	20.54%
`__printf_fp_buffer_1.isra.0`	150,473,022	9.05%
`____strtod_l_internal`	110,896,754	6.67%
`std::num_get<...>::_M_extract_float(...) const`	85,195,322	5.12%

Figure 9. Optimization 1 pearson valgrind results

Moving normalization outside the pairwise loop removed the main baseline hotspots: **Vector::operator-()** and **Vector::operator/(),** which previously accounted for 42.84% of instructions**. mean(), magnitude(), and normalize()** also disappeared from the top 10 hotspots. **Vector::dot(Vector) const** then became the largest hotspot at 24.25%, followed by the copy constructor (**Vector::Vector(Vector const&)**) at 20.54% because its argument was still passed by value.

Optimization 2

Blur

After Optimization 1, the profile showed Matrix::r/g/b accessors and the loop-bound queries as the next costs. Looking at the loop conditions in the pixel loops, get_x_size() and get_y_size() simply return x_size. The values never change during blur.

Therefore, optimization 2, using loop-invariant hoisting of the loop bounds optimization technique, fetches each dimension once(makes them as variables) before the loops and reuses the cached values. Six call sites replaced with two variable reads.

Image	Opt 1 (s)	Opt 2 (s)	Speedup
im1.ppm	0.153	0.114	1.34×
im2.ppm	0.325	0.243	1.34×
im3.ppm	0.527	0.423	1.24×
im4.ppm	2.680	2.120	1.26×

Figure 10. optimization 1 vs optimization 2 speedup

Average median wall time is 1.29x faster than Opt 1, consistent across all four images. Output byte-identical to baseline.

Function	Ir	Dr	Dw	% Ir
`Filter::blur`	2,629,913,867	906,123,343	526,898,208	61.07%
`Matrix::r(unsigned, unsigned)`	333,086,720	199,852,032	0	7.73%
`Matrix::b(unsigned, unsigned)`	333,086,720	199,852,032	0	7.73%
`Matrix::g(unsigned, unsigned)`	333,086,720	199,852,032	0	7.73%

Figure 11. optimization 2 callgrind profiler results

The improvement is real and consistent. Matrix::get_x_size and get_y_size do not appear in the top rows of this Opt 2 profile, their call counts collapsed from hundreds of millions (in Opt 1) to a handful and , therefore, are not shown on top rows of callgrind profiler.

Pearson

As seen in Figure 9 Optimization 1 made Vector::dot(Vector) const the largest application hotspot at 24.25% of the complete profile. The copy constructor (Vector::Vector(Vector const&))was second at 20.54%. Inspection of dot() showed why these costs appeared together: its right hand operand was passed by value.

Optimization 2 iterates on optimization 1 changes so the second optimization was applied on top of the first optimizations.

Passing rhs in function double vector::dot(Vector rhs) const by value creates and destroys a temporary vector for every dot() call. Since 512.data contains 130,816 unique pairs, this causes unnecessary deep copies. Optimization 2 eliminates these copies by changing the parameter from pass-by-value to pass-by-constant-reference.

This **copy elimination optimization** passes rhs by constant reference, avoiding a new vector while preventing modification. The dot-product calculation and results remain unchanged.

Input	Opt. 1 executions	Opt. 2 executions	Opt. 1 mean (s)	Opt. 2 mean (s)	Opt. 1 median (s)	Opt. 2 median (s)	Mean speedup	Median speedup
`128.data`	1,000	1,000	0.01519	0.01475	0.01540	0.01280	1.03x	1.20x
`256.data`	200	200	0.04915	0.04980	0.04825	0.04800	0.99x	1.01x
`512.data`	60	120	0.20883	0.16642	0.19917	0.16583	1.25x	1.20x
`1024.data`	10	20	0.98600	0.84050	0.94000	0.77000	1.17x	1.22x

Figure 12. Comparison of optimization 1 against optimization 2 mean and median timings

Optimization 2 was compared directly with Optimization 1. The larger workloads provide the clearest timing evidence. For 512.data, mean and median speedups were 1.25x and 1.20x; for 1024.data, they were 1.17x and 1.22x, confirming improvement on larger workloads. For 256.data, the mean showed a 1.32% slowdown while the median showed a 1.01x speedup, indicating timing noise rather than a meaningful regression. Different execution counts resulted from automatic batch calibration, and all times were normalized per execution.

Input	Baseline executions	Opt. 2 executions	Baseline mean (s)	Opt. 2 mean (s)	Baseline median (s)	Opt. 2 median (s)	Mean speedup	Median speedup
`128.data`	1,000	1,000	0.01972	0.01475	0.01930	0.01280	1.34x	1.51x
`256.data`	200	200	0.09430	0.04980	0.09325	0.04800	1.89x	1.94x
`512.data`	15	120	0.57267	0.16642	0.57667	0.16583	3.44x	3.48x
`1024.data`	5	20	3.58000	0.84050	3.56000	0.77000	4.26x	4.62x

Figure 13. Comparison of optimization 2 against baseline mean and median timings

Compared with the baseline on 1024.data, Optimization 2 reduced mean runtime from 3.580 to 0.841 seconds (4.26x speedup) and median runtime from 3.560 to 0.770 seconds (4.62x). One slower batch increased the optimized mean, explaining the higher median speedup. Execution counts differed because of automatic calibration but normalized per-execution times remained comparable.

Function reported by Callgrind	Instructions	Share of all instructions
`Vector::dot(Vector const&) const`	403,436,544	31.52%
`__printf_fp_buffer_1.isra.0`	150,473,022	11.76%
`____strtod_l_internal`	110,896,754	8.66%
`std::num_get<...>::_M_extract_float(...) const`	85,195,322	6.66%
`__mpn_mul_1`	56,819,529	4.44%

Figure 14 Calgrind Top 10 function calls for pearson optimization 2

After Optimization 2, Vector::dot(Vector const&) const became the main hotspot, increasing from 24.25% to 31.52% of total instructions. This relative increase does not mean it became slower: it still executed 403,436,544 instructions because its multiplication and accumulation loop was unchanged. Passing the right-hand vector by constant reference instead removed calls to Vector::Vector(Vector const&), reducing the total program work. The unchanged dot products therefore represent a larger share of the smaller program and remain necessary for every vector pair.

Optimization 3

Blur

After Optimizations 2, the Callgrind profile (Figure 11) showed the pixel loops dominating. The three accessors alone accounted for 23.2% of the program — 999,260,160 instructions, computing R[y * x_size + x] for one pixel each. Removing them removes nearly a quarter of the runtime. Two problems remained. Every pixel read went through Matrix::r/g/b — a multiply, an add, and a memory load per access. And the loop order was cache-hostile: with x outer and y inner, each iteration jumped a full row ahead in memory.

We fixed both in one step because they target the same cost, the per-pixel access pattern. The two techniques are removing unneeded function calls and loop interchange for cache locality.
For the accessors, Matrix already exposes get_R(), get_G(), get_B(). We grab those once and index into the raw buffers directly, replacing the call with a single array lookup. For the loop order, we swapped x and y so the inner loop reads consecutive bytes — a cache line is fetched once and reused for 64 pixels instead of being thrown away every step.

Image	Opt 2 (s)	Opt 3 (s)	Speedup
im1.ppm	0.114	0.048	**2.38×**
im2.ppm	0.243	0.161	**1.51×**
im3.ppm	0.423	0.170	**2.49×**
im4.ppm	2.120	0.940	**2.26×**

Figure 15. optimization 2 vs optimization 3 speedup

Function	Ir	Dr	Dw	% Ir
`Filter::blur`	1,500,315,953	234,457,177	8,390,699	68.90%
`std::basic_streambuf::xsgetn`	125,829,131	28,311,559	25,165,825	5.78%
`std::istream::read`	122,683,392	44,040,192	28,311,552	5.63%
`std::ostream::put`	116,393,199	53,478,195	22,019,733	5.35%
`std::istream::sentry`	97,517,661	31,457,310	22,020,117	4.48%

Figure 16. optimization 3 callgrind profiler results

Based on figure 16, three accessor functions are gone from the top of the profile. In Opt 2, Matrix::r, Matrix::g, and Matrix::b each executed 333,086,720 instructions — 22% of the program combined. In Opt 3, none of them appear in the top rows at all. They've been replaced by direct array indexing, which the compiler folds into the surrounding loop. Filter::blur dropped by 43%. From 2,629,913,867 to 1,500,315,953 instructions. Average median wall time ~2.1x faster than Optimization 2,based on Figure 15. Output is byte-identical to the baseline on all four images.

Parallelization

Blur

We parallelized the baseline Filter::blur with pthreads. The easiest way to split the work is by row bands. In the horizontal pass, the output pixel scratch(x, y) only depends on pixels from row y of the input. In the vertical pass, the output pixel dst(x, y) only depends on pixels from column x of scratch. Because each output row can be computed independently, we divided the rows [0, h_size) into N contiguous bands and gave each thread one band. No two threads write to the same row, so no locks or mutexes are needed.

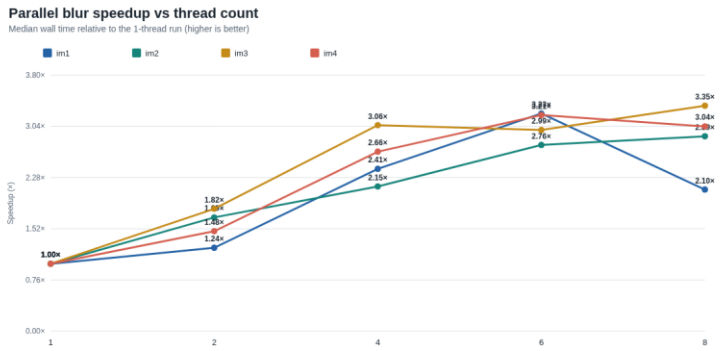


Figure 17. parallel blur speedup vs thread count

Parallelizing the baseline blur gives a consistent 2.5–3.3x speedup on larger images. Speedup is real but sub-linear. Peak speedup is around 3× on the largest images with 4–6 threads, well below the theoretical 6–8x. The CPU cores saturate the memory bus before they saturate the ALUs. Im1 reaches its best at 6 threads (3.23×) and drops to 2.10x at 8. Im4 keeps improving to 6 threads before flattening.

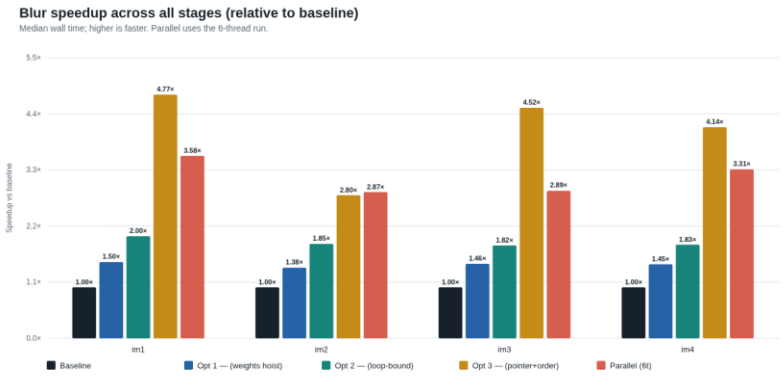


Figure 18. Blur speedup across all stages relative to baseline

Across the five stages, blur went from 3.89 s on im4 at baseline to 0.94 s with the sequential optimizations (≈4.1x) and to 1.18 s at 6 threads with pthreads on top of baseline (~3.2x). On im1 the trend reverses: speedup climbs from 1.24x (2 threads) to 3.23x (6 threads) and then drops to 2.10x at 8 threads, showing that thread overhead outweighs parallel work once per-thread workload becomes small.

Pearson

Pearson pair calculations are independent, so the pairs are divided into nearly equal sections and assigned to worker threads. Each worker processes one section using the original pearson() function, while Linux schedules the threads across available CPUs without explicit core binding.

Workers read the shared datasets without modifying them and write to separate positions in a preallocated result vector. This preserves sequential output order and avoids the need for a mutex.

Parallel Pearson runtime and speedup by thread count

Runtime per execution; blue values show speedup relative to the one-thread run

Median runtime and speedup

Threads	128.data	256.data	512.data	1024.data
1	0.018700 s 1.00x	0.098000 s 1.00x	0.541667 s 1.00x	3.620000 s 1.00x
2	0.017350 s 1.08x	0.075750 s 1.29x	0.390000 s 1.39x	2.290000 s 1.58x
4	0.018900 s 0.99x	0.067000 s 1.46x	0.259167 s 2.09x	1.335000 s 2.71x
6	0.017950 s 1.04x	0.059000 s 1.66x	0.222500 s 2.43x	1.050000 s 3.45x
8	0.020000 s 0.94x	0.056250 s 1.74x	0.228333 s 2.37x	1.030000 s 3.51x

Mean runtime and speedup

Threads	128.data	256.data	512.data	1024.data
1	0.019960 s 1.00x	0.114300 s 1.00x	0.552667 s 1.00x	3.642000 s 1.00x
2	0.019950 s 1.00x	0.075600 s 1.51x	0.403333 s 1.37x	2.302000 s 1.58x
4	0.020540 s 0.97x	0.070300 s 1.63x	0.256167 s 2.16x	1.310000 s 2.78x
6	0.017650 s 1.13x	0.059400 s 1.92x	0.251500 s 2.20x	1.027000 s 3.55x
8	0.021980 s 0.91x	0.076750 s 1.49x	0.233333 s 2.37x	1.088000 s 3.35x

The main thread waits for all workers using pthread_join().

Figure 19.Parallelized pearson Mean and median runtimes over each dataset and thread count

Using the median runtime the largest workload scaled most consistently: speedup increased from 1.58x at two threads to 2.71x at four, 3.45x at six, and 3.51x at eight. Smaller workloads did not contain enough computation to outweigh the costs of creating and joining threads, allocating memory, parsing input, and writing output. Eight threads slowed 128.data to 0.94x. The best

measured 256.data median occurred at eight threads (1.74x), while the best 512.data median occurred at six threads (2.43x) and regressed slightly at eight threads (2.37x).

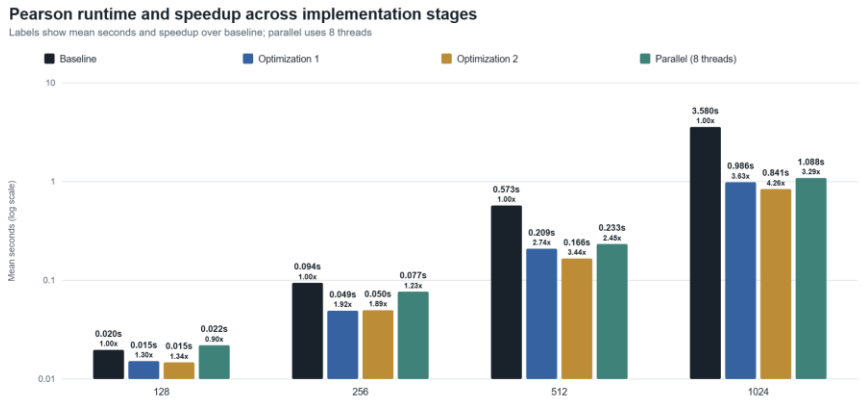


Figure 20. Pearson mean runtime and speedup across all implementation stages relative to the sequential baseline.

Runtime improvements increase with dataset size. On 1024.data, Optimization 1 reduces runtime from 3.580 to 0.986 seconds (3.63x), Optimization 2 to 0.841 seconds (4.26x), and the eight-thread parallel version to 1.088 seconds (3.29x). Optimization 2 performs best because removing repeated normalization and vector copying saves more time than parallelizing the unchanged baseline. For 128.data, the parallel version slows to 0.90x because thread overhead exceeds the time saved. Overall, source optimization provides the best improvements because they remove work from the algorithm.

Conclusion

Profiling identified repeated computation and unnecessary copying as the main performance problems. Blur was optimized by calculating Gaussian weights once, caching loop bounds, replacing RGB accessor calls with direct indexing, and improving loop order. On im4.ppm, these changes reduced median runtime from 3.89 to 0.94 seconds, approximately 4.1x faster. Pearson was optimized by normalizing each vector once before the pairwise loop and passing the dot() operand by constant reference. On 1024.data, its mean runtime fell from 3.580 to 0.841 seconds, producing a 4.26x speedup.

The baseline versions were also parallelized independently using POSIX threads (pthreads). Blur rows and Pearson dataset pairs were divided into nearly equal sections and assigned to workers, with each worker writing to separate output positions. Parallel Blur reached approximately 3.2x speedup on im4.ppm, while eight-thread Pearson achieved 3.29x over the original baseline on 1024.data. Smaller workloads gained less because thread creation, joining, and scheduling costs formed a larger part of their runtime.

All sequentially optimized and parallel outputs passed correctness checks, and representative Helgrind tests reported no threading errors. Overall, removing redundant work produced the greatest improvement, while parallelization was most effective for larger workloads. Combining the sequential optimizations with multithreading would be the logical next step.